AI Notes Summarizer

Project Interview Preparation Notes

Personal cheat sheet aligned to the actual ai-notes codebase and resume.

Resume claims covered:

- Notes CRUD + PDF uploads
- Groq LLM summaries and quizzes
- Firebase Authentication + Firestore user-specific storage
- Stack: React, Firebase, Groq LLM, Netlify Functions

How to use this PDF

Revise sections 1-3 and 20 before every interview. Use sections 5-11 for deep follow-ups. Anything marked CHECK BEFORE INTERVIEW must be verified in your Firebase/Netlify console.

1. Project Overview (1-minute revision)

AI Notes Summarizer

One-liner: A web app where users manage personal notes and PDFs, then generate AI summaries and quizzes using Groq LLM, with Firebase Auth and Firestore for user-specific storage.

Problem it solves

- Students/professionals have long notes and PDFs and need quick summaries + practice quizzes.

Main features

- Email/password + Google login (Firebase Auth)
- Create / edit / delete notes (Firestore)
- Favorites, search, analytics heatmap
- PDF upload with client-side text extraction (pdfjs-dist)
- AI summary via Netlify Function -> Groq
- AI quiz (5 MCQs) via Netlify Function -> Groq

Tech stack and why

- React (Vite): Component UI, routing, SPA.
- Firebase Auth: Ready-made secure auth (email + Google).
- Firestore: Cloud NoSQL for notes CRUD + realtime listener.
- Groq LLM (llama-3.1-8b-instant): Fast, cheap summarization + quiz generation.
- Netlify Functions: Serverless proxy so GROQ_API_KEY stays off the frontend.
- pdfjs-dist: Extract text from PDFs in the browser before sending to AI.

2. 30-Second Interview Explanation

Speak this

I built AI Notes Summarizer, a React web app that helps users manage study notes and PDFs. Users can create, edit, and delete notes, upload PDFs, and generate AI summaries and quizzes. I used Firebase Authentication so each user has a private account, and Firestore to store their notes. For AI, I call Groq's LLM through Netlify Functions so the API key stays on the server. The model returns bullet-point summaries and five multiple-choice quiz questions from the note content.

3. 1-2 Minute Detailed Explanation

When an interviewer asks for more detail, walk the full flow:

1. User opens the React SPA (Vite) and signs in with email/password or Google via Firebase Auth.
2. App.jsx listens with onAuthStateChanged; ProtectedRoute blocks /dashboard, /favorites, /analytics, /settings if not logged in.

3. On Dashboard, notes load in realtime via subscribeToUserNotes(userEmail) from Firestore collection notes.
4. User creates/edits/deletes notes through createNote / updateNote / deleteNote in firebaseDB.js.
5. For PDFs: browser uses pdfjs-dist to extract text (max 5MB, 30 pages), chunks text (~600 chars), summarizes each chunk, and stores truncated content + page summaries.
6. For AI summarize/quiz: frontend POSTs to /.netlify/functions/summarize or generateQuiz. The function calls Groq chat completions with llama-3.1-8b-instant using GROQ_API_KEY from env.
7. Summary/quiz is written back to the note document (updateSummary / updateQuiz) and shown in the UI.

4. Complete Project Architecture

```
Browser (React SPA)
  |-- Auth UI (Login.jsx) -----> Firebase Authentication
  |-- ProtectedRoute / App.jsx -> auth state (onAuthStateChanged)
  |-- Dashboard / Favorites / Analytics / Settings
  |-- firebaseDB.js ----------> Firestore (collection: notes)
  |-- pdfjs-dist -------------> extract PDF text in browser
  |-- fetch('/.netlify/functions/...')
         |
         v
Netlify Functions (summarize.js, generateQuiz.js)
         |
         v
Groq API  (model: llama-3.1-8b-instant)
         |
         v
Response -> frontend -> Firestore update -> UI
```

Responsibilities

- React: UI, forms, PDF upload, calling functions, displaying summary/quiz.
- Firebase Auth: identity (user.email used as ownership key).
- Firestore: persistent notes per userEmail; realtime onSnapshot.
- pdfjs-dist: turn PDF into text before LLM (LLM needs text, not binary PDF).
- Netlify Functions: hide API key; call Groq; return summary/quiz JSON.
- Groq: generate summary bullets or 5 MCQ JSON.

Not in this project

No custom Node/Express server. No MongoDB. No Hugging Face usage in code (VITE_HF_API_KEY may exist in .env but is unused). No Firebase Storage for PDF files - only extracted text (truncated) is stored.

5. End-to-End Flow

A. User Registration / Login

1. Register: Login.jsx handleSignup -> createUserWithEmailAndPassword (password min 6). Name field is collected in UI but not saved via updateProfile.
2. Login: handleLogin -> signInWithEmailAndPassword. Google: handleGoogleLogin -> signInWithPopup(googleProvider).
3. Forgot password: sendPasswordResetEmail.
4. Identity: auth.currentUser / user.email. App.jsx stores isLoggedIn + userEmail in localStorage for convenience; real source of truth is Firebase Auth.
5. Auth state: onAuthStateChanged in App.jsx and ProtectedRoute.jsx.
6. Logout: signOut(auth) from TopNavbar (also available in Settings).
7. Routes: protected pages redirect to /auth if no user.

Note

App.jsx can complete email magic-link sign-in (signInWithEmailLink), but Login never sends the link (sendSignInLinkToEmail missing). Do not claim magic-link signup unless you add it.

B. Creating a Note

1. User fills title + content on Dashboard (showNewNote form).
2. handleAddNote validates non-empty title and content (early return if empty).
3. createNote(userEmail, title, content) in firebaseDB.js -> addDoc to notes with userEmail, isFavorite:false, summary:null, quiz:null, date, createdAt, updatedAt.
4. Association key: userEmail (string from auth user), not Firebase uid.

5. UI updates via realtime subscribeToUserNotes onSnapshot - no manual list refresh needed.

C. Editing a Note

User enters edit mode (isEditing / editingId). handleAddNote calls updateNote(editingId, { title, content, summary: null }). Summary is cleared so the user re-generates AI summary after edits. updatedAt is set in updateNote.

D. Deleting a Note

handleDeleteNote(id) -> deleteNote(id) -> deleteDoc. Clears selectedNote in UI. Errors show alert.

E. PDF Upload

1. Hidden file input accept=application/pdf -> handlePDFUpload.
2. Reject if > 5MB. extractTextFromPDF rejects if > 30 pages.
3. pdfjs-dist getDocument + getTextContent per page; concatenates item.str.
4. chunkText(text, 600) then summarizeWithAI per chunk -> pageSummaries [{page, text}].
5. createNote(userEmail, file.name, text.slice(0,2000), pageSummaries). Full text is NOT fully stored - only first 2000 chars in content.
6. Later quizzes use note.content (the truncated text).

F. AI Summary Generation

```
note.content -> summarizeWithAI (Dashboard/Favorites)
  POST /.netlify/functions/summarize  body: { text }
  Netlify handler -> Groq chat/completions model llama-3.1-8b-instant
  returns { summary } -> updateSummary(noteId, summary) -> Firestore -> UI
```

- API key: process.env.GROQ_API_KEY in Netlify Functions (not VITE_*).
- Why not frontend: any VITE_ key is bundled into browser JS and can be stolen.
- Model: llama-3.1-8b-instant; temperature 0.3; max_tokens 250.

G. AI Quiz Generation

```
note.content -> generateQuizWithAI
  POST /.netlify/functions/generateQuiz  body: { text }
  System prompt: EXACTLY 5 MCQs, JSON only [{question, options[4], answer}]
  Function JSON.parse(model content) -> returns array
  Frontend checks Array.isArray -> updateQuiz(noteId, quiz)
  UI lists questions with options and revealed answer (not interactive scoring)
```

- Model same; temperature 0.4; max_tokens 700.
- If model returns markdown fences, JSON.parse fails -> alert Quiz generation failed.

6. Important Files and Functions

File	Function / Component	Purpose	Remember
src/App.jsx	onAuthStateChanged + routes	Auth gate + routing	Source of login state
src/auth/Login.jsx	handleLogin/Signup/Google	Auth UI	Firebase Auth methods
src/layout/ProtectedRoute.jsx	ProtectedRoute	Blocks unauthenticated pages	Navigate to /auth
src/firebase.js	auth, db, googleProvider	Firebase init + persistence	VITE_FIREBASE_* env
src/firebaseDB.js	create/update/deleteNote, subscribe...	All Firestore CRUD	notes + userEmail
src/pages/Dashboard.jsx	CRUD + PDF + AI	Main feature page	Most interview depth
src/pages/Favorites.jsx	favorites + summarize	Favorite notes view	Filter isFavorite
src/pages/Analytics.jsx	stats + heatmap	Usage analytics	Uses subscribeToUserNotes
src/pages/Settings.jsx	password + logout	Account settings	Reauth for password
src/components/TopNavbar.jsx	nav + handleLogout	Navigation	signOut(auth)
netlify/functions/summarize.js	handler	Groq summary proxy	GROQ_API_KEY server-side
netlify/functions/generateQuiz.js	handler	Groq quiz proxy	JSON array response
netlify.toml	functions=...	Functions folder + SPA redirect	Need netlify dev locally

7. Firebase Authentication

What: Managed auth service. How: Email/password + Google popup; App listens with onAuthStateChanged; ProtectedRoute wraps private pages. Why: Avoid building password hashing, sessions, OAuth yourself.

- User ID used for data: primarily user.email stored as userEmail on notes.
- Logout clears Firebase session and localStorage flags.

Why Firebase Auth instead of building yourself?

Building auth securely is hard (hashing, resets, Google OAuth, session theft). Firebase gives email and Google login quickly, integrates with Firestore rules, and fits a student/solo project timeline.

8. Firestore Database Design

Actual structure (flat collection - not nested users/{uid}/notes)

```
notes/{autoDocId}
  userEmail: string   <-- ownership key
  title: string
  content: string     (PDF: first 2000 chars only)
  isFavorite: boolean
  summary: null | string | [{page, text}] for PDFs
  quiz: null | [{question, options[], answer}]
  date: YYYY-MM-DD
  createdAt, updatedAt
```

- Read: query where userEmail == current user; sort createdAt client-side.
- Realtime: onSnapshot in subscribeToUserNotes.

Why Firestore?

Realtime updates, easy Firebase Auth integration, no server to manage, document model fits notes well. For this app's scale and CRUD shape, Firestore is simpler than running MongoDB myself.

CHECK BEFORE INTERVIEW

No firestore.rules file is in the repo. Recommended rules are documented in FIRESTORE_SETUP.md (email on token must match userEmail). Verify in Firebase Console that rules are published. If still in open test mode, say so honestly and mention tightening rules as a next step.

9. Groq LLM Integration

What: Fast LLM API (OpenAI-compatible chat completions). How: Netlify Functions call https://api.groq.com/openai/v1/chat/completions with model llama-3.1-8b-instant. Why: Low latency and free/cheap tier good for demos; simple HTTP integration.

- Summary prompt: 4-6 concise bullet points, simple language, no headings.
- Quiz prompt: exactly 5 MCQs, JSON only, options A-D style array, answer field.
- Frontend never sees GROQ_API_KEY; only receives summary string or quiz array.

Interview phrasing

I keep the Groq key in Netlify environment variables. React sends the note text to my serverless function, the function attaches the API key and calls Groq, then returns the result. That way the key is never shipped in the client bundle.

10. Netlify Functions

What: Serverless Node handlers deployed with the site. How: netlify/functions/summarize.js and generateQuiz.js export handler(event). React fetch POSTs JSON { text }. Why: Protect secrets and avoid running a full Express server.

- summarize: validates key + text; returns { summary }.
- generateQuiz: same; parses model JSON; returns quiz array.
- Local: need netlify dev (plain vite alone won't serve /.netlify/functions).

Why not call Groq directly from React?

Because the API key would be visible in DevTools/network and anyone could steal quota. A Netlify Function acts as a thin backend that holds the secret.

Honest limitation

Functions currently have no auth check - anyone who can hit the URL could burn Groq credits. Improvement: verify Firebase ID token in the function before calling Groq.

11. PDF Processing

```
Upload PDF -> pdfjs-dist extract text per page
-> chunk (~600 chars) -> summarize each chunk via Groq
-> save note: title=filename, content=first 2000 chars, summary=[{page,text}]
```

- Library: pdfjs-dist (browser). Limits: 5MB, 30 pages.
- Why extract first: LLMs need text tokens; you cannot usefully POST a raw PDF binary to this chat API flow.
- Scanned/image-only PDFs: text extraction returns little/empty - relevant limitation to mention if asked.
- Quiz later uses truncated content (2000 chars), not the full PDF text.

12. Important Technical Decisions - Why did I choose...?

Why React?

Component model and fast SPA UI for forms, lists, and AI results. Vite keeps DX simple.

Why Firebase?

Auth + DB in one platform; quick to ship without managing servers.

Why Firestore?

Realtime listeners and document CRUD map cleanly to notes; works with Firebase Auth.

Why Groq?

Fast inference, OpenAI-compatible API, good for low-latency summaries/quizzes.

Why Netlify Functions?

Deploy frontend + API together; keep GROQ_API_KEY server-side without Express.

Why not MongoDB?

Would need my own backend/hosting; Firestore already covers CRUD + auth linkage.

Why not Node/Express?

Overkill for two AI endpoints; serverless is enough and cheaper to operate.

Why use an LLM?

Summaries and quiz generation are language tasks - LLMs do this better than regex/rules.

Why quizzes not only summaries?

Active recall helps studying; MCQs are a natural second LLM use case.

Why userEmail not uid?

Simple ownership field matching auth.email; works with documented rules. uid is more stable if email changes - possible improvement.

13. Security Questions

Q: Where is the Groq API key stored?
Answer: In Netlify environment variable GROQ_API_KEY, read only inside Netlify Functions via process.env.

Q: Why shouldn't API keys be in React?
Answer: Frontend code is public. Anyone can extract a VITE_ key from the bundle and abuse the API.

Q: How do users only access their own notes?
Answer: Queries filter where userEmail == logged-in email. Real enforcement should be Firestore security rules matching auth token email. CHECK deployed rules before interview.

Q: How does Firebase Auth help?
Answer: It proves who the user is. Combined with rules, only that identity can read/write matching notes.

Q: What security rules does Firestore use?
Answer: Recommended rules are in FIRESTORE_SETUP.md. No rules file is committed. Say: I document email-based ownership rules; I will confirm they are published in the console.

Q: What would you improve security-wise?
Answer: Confirm/publish Firestore rules; verify Firebase ID token inside Netlify Functions; rate-limit AI endpoints; stop relying on localStorage flags as security; prefer uid over email.

14. Performance and Scalability

What my project currently does

- Client-side PDF extraction and sequential chunk summarization (can be slow for large PDFs).
- Realtime note listener per logged-in user.
- AI calls on demand (no caching of summaries beyond storing on the note document).
- Alerts on AI failure; no automatic retry/backoff.
- PDF capped at 5MB / 30 pages; content stored truncated to 2000 chars.

What I would improve in production

10,000 users?

Firestore scales for this pattern; watch read costs from onSnapshot. Add pagination, indexes, CDN for static assets.

Very large PDF?

Stricter limits, server-side extraction, async job queue, store chunks, summarize in background.

Reduce Groq usage?

Don't re-summarize if summary exists; shorter prompts; smaller model for drafts; cache by content hash.

Cache summaries?

Already stored on note; add contentHash so edits invalidate; optional Redis/CDN for function responses.

Rate limits?

Per-user quotas in functions using Auth UID; exponential backoff; queue.

Faster responses?

Parallel chunk summaries carefully; stream tokens; pre-chunk on upload.

AI API fails?

Show clear error, retry button, keep note usable without AI; circuit breaker.

Concurrent requests?

Disable button while aiLoading (already); server-side concurrency limits.

15. Error Handling

What exists today

- Auth: Firebase error codes mapped to user-facing alerts in Login.jsx.
- Notes CRUD: try/catch + alert + console.error.
- PDF: size/page checks; alert on failure; progress UI while processing.
- Summarize/Quiz: alert on failure; summarize checks data.summary; quiz checks Array.isArray.
- Functions: 400 missing text; 500 missing key / exceptions; forward Groq errors.
- Loading: auth spinner; notes hydration; aiLoading disables buttons; PDF progress bar.
- Empty note create: early return if title/content missing.

Possible improvements

- Toasts instead of alert(); retry for transient network errors.
- Resync selectedNote after Firestore updates (possible stale UI after summarize/quiz).
- Validate quiz schema fields, not only Array.isArray.
- Strip markdown from model output before JSON.parse.
- Global error boundary in React.

16. Interview Questions & Answers (25+)

Basic

1. What is your project?
Answer: AI Notes Summarizer - a React web app for managing notes and PDFs with AI summaries and quizzes, using Firebase and Groq via Netlify Functions.

2. Why did you build it?
Answer: I wanted a practical full-stack project that combines CRUD, auth, file handling, and real LLM integration for studying.

3. What technologies did you use?
Answer: React with Vite, Firebase Auth, Firestore, pdfjs-dist, Groq LLM, and Netlify Functions. Bootstrap/Font Awesome for UI.

4. What is the role of React?
Answer: It renders the UI, manages local state for forms and selected notes, handles routing, and calls Firebase and Netlify endpoints.

5. What is Firebase?
Answer: A Google backend platform. I used Authentication for login and Firestore as the cloud database for notes.

Medium

6. How does authentication work?
Answer: Users sign up or log in with email/password or Google. Firebase issues a session. App.jsx uses onAuthStateChanged, and ProtectedRoute blocks private pages.

7. How does Firestore store notes?
Answer: Flat collection notes. Each document has userEmail, title, content, summary, quiz, favorites, and timestamps.

8. How does CRUD work?
Answer: createNote addDoc, updateNote updateDoc, deleteNote deleteDoc, subscribeToUserNotes onSnapshot. Dashboard handlers call these.

9. How does PDF upload work?
Answer: Browser extracts text with pdfjs-dist, chunks it, summarizes chunks with Groq, saves truncated text plus page summaries to Firestore.

10. How does the Groq API work?
Answer: OpenAI-compatible chat completions. My function sends system + user messages and reads choices[0].message.content.

11. Why use Netlify Functions?
Answer: To keep GROQ_API_KEY secret and host two small AI endpoints next to the static React app.

12. How do you protect the API key?
Answer: It lives in Netlify env as GROQ_API_KEY. Only serverless code reads it. Not prefixed with VITE_.

13. How is the AI prompt created?
Answer: Fixed system prompts in summarize.js and generateQuiz.js. User message is the note/PDF text.

14. How is the quiz generated?
Answer: generateQuiz asks for exactly 5 MCQs as JSON. Function parses JSON and frontend stores the array on the note.

15. How does frontend talk to serverless?

Answer: fetch POST to /.netlify/functions/summarize or generateQuiz with JSON body { text }.

Deep

16. Why Firestore instead of MongoDB?
Answer: I didn't want to host a separate DB and API. Firestore gives realtime queries and ties into Firebase Auth with less ops work.

17. Why Groq instead of another LLM?
Answer: Speed and simple HTTP API. llama-3.1-8b-instant is enough for short summaries and quizzes.

18. Why serverless functions?
Answer: Only needed a secure proxy for Groq - not a full backend. Pay for usage and deploy with the site.

19. How would you scale this?
Answer: Pagination, stricter PDF limits, background jobs for AI, function auth + quotas, confirm Firestore rules, maybe move extraction server-side.

20. What if Groq is unavailable?
Answer: Today the UI alerts failure and the note remains. Production: retries, fallback model, queue, and cached previous summaries.

21. How reduce API costs?
Answer: Skip regenerate if summary exists; truncate input; lower max_tokens; cache by hash; rate-limit users.

22. How handle huge PDFs?
Answer: Current hard limits help. Next: async processing, store full text in Storage, summarize in jobs, warn on scanned PDFs.

23. How improve security?
Answer: Publish Firestore rules, verify ID tokens in functions, rate limits, use uid, remove unused env keys.

24. What was the hardest part?
Answer: Keeping AI key safe while still calling Groq from a SPA, plus PDF text extraction/chunking so summaries stay useful within token limits.

25. What would you improve with more time?
Answer: Function auth, robust quiz JSON parsing, interactive quiz scoring, full PDF text storage strategy, and fix selectedNote stale UI after AI updates.

26. Why associate notes by email not uid?
Answer: It matched my rules docs and was simple. uid is better long-term because emails can change.

27. Do you store the PDF file?
Answer: No. I extract text client-side and store truncated content plus summaries. No Firebase Storage upload in this codebase.

28. Why chunk PDF text?
Answer: Long PDFs exceed practical prompt size. Chunking summarizes sections then stores page-wise summaries.

29. How do Favorites work?
Answer: toggleFavorite flips isFavorite. Favorites page filters notes where isFavorite is true.

30. Why edit clears summary?
Answer: So outdated AI summaries don't stay after content changes; user regenerates intentionally.

17. Hardest Part of the Project

1) Protecting the Groq API key in a SPA

Problem -> What I did -> Why -> Result

Problem: React alone would expose any client-side key. What I did: Added Netlify Functions summarize and generateQuiz that read GROQ_API_KEY from env. Why: Secrets must stay server-side. Result: Frontend only sends text and receives summary/quiz.

2) PDF -> useful AI input under limits

Problem: PDFs can be long; models have token limits; raw PDF isn't text. What I did: pdfjs-dist extraction, 5MB/30 page caps, chunkText(600), summarize chunks, store 2000-char content. Why: Keep requests reliable and costs bounded. Result: Upload flow works for typical study PDFs; tradeoff is truncated content for later quizzes.

3) Structured quiz output from an LLM

Problem: Models may return markdown or invalid JSON. What I did: Strict system prompt (JSON only, exact schema) + JSON.parse in the function + Array.isArray check in React. Why: UI needs predictable objects. Result: Works when model obeys; still fragile - good follow-up improvement topic.

18. What I Personally Worked On

Frontend

- React pages: Login, Dashboard, Favorites, Analytics, Settings; routing and ProtectedRoute.
- Notes UI, search, favorites toggle, analytics heatmap, loading/PDF progress states.

Firebase

- Auth flows (email/password, Google, reset, logout) and Firestore helpers in firebaseDB.js.

AI integration

- Netlify Functions calling Groq; summary + quiz prompts; saving results on notes.

PDF functionality

- Client-side extraction with pdfjs-dist, chunking, and creating notes from uploads.

Backend / serverless

- summarize.js and generateQuiz.js; netlify.toml functions config.

Do not claim a custom Express API or MongoDB - not in this codebase.

19. Resume Alignment

Bullet: "create, edit, delete, and manage personal notes along with PDF uploads"

Explain: Dashboard createNote/updateNote/deleteNote; favorites and search as manage; PDF via handlePDFUpload + pdfjs-dist. Evidence: firebaseDB.js + Dashboard.jsx.

Bullet: "Integrated Groq LLM to generate AI-powered summaries and quizzes"

Explain: Not direct from browser - Netlify Functions call Groq llama-3.1-8b-instant. Summaries for notes and PDF chunks; quizzes are 5 MCQs stored on the note. Evidence: summarize.js, generateQuiz.js, Dashboard summarize/quiz handlers.

Bullet: "Firebase Authentication and Firestore for user-specific note storage and CRUD"

Explain: Login/signup/Google; onAuthStateChanged; notes filtered by userEmail; CRUD helpers and realtime subscription. Evidence: Login.jsx, App.jsx, firebaseDB.js.

Extra features (say only if asked)

- Analytics heatmap, favorites page, settings password change - implemented but not in resume bullets.

20. Interview Explanation Cheat Sheet

Project: AI Notes Summarizer

Stack: React + Firebase Auth + Firestore + Groq LLM + Netlify Functions (+ pdfjs-dist)

Core features

- Notes CRUD
- PDF upload/extract
- AI summary
- AI quiz
- Auth
- User-specific Firestore notes

Architecture

```
React -> Firebase Auth/Firestore
React -> Netlify Functions -> Groq
React (pdfjs) -> text -> AI -> Firestore
```

Important files

App.jsx, Login.jsx, ProtectedRoute.jsx, firebase.js, firebaseDB.js, Dashboard.jsx, netlify/functions/summarize.js, generateQuiz.js

Important functions

createNote, updateNote, deleteNote, subscribeToUserNotes, updateSummary, updateQuiz, handlePDFUpload, extractTextFromPDF, summarizeWithAI, generateQuizWithAI, handler (both functions)

Key concepts

onAuthStateChanged - userEmail ownership - onSnapshot realtime - serverless API key proxy - PDF text extraction before LLM - prompt engineering for JSON quizzes - client limits 5MB/30 pages

30-second script

I built a React notes app with Firebase Auth and Firestore so each user has private notes. Users can upload PDFs, extract text in the browser, and generate Groq AI summaries and quizzes through Netlify Functions so the API key stays server-side.

5 most likely questions

What does your project do?

Answer: Notes + PDF management with AI summary and quiz, secured by Firebase, AI via Groq/Netlify.

How is the API key safe?

Answer: GROQ_API_KEY only in Netlify env; React never sees it.

How is data user-specific?

Answer: Notes store userEmail; queries filter by logged-in email; rules should enforce match.

How do PDFs work?

Answer: pdfjs extracts text -> chunk -> summarize -> save truncated content + summaries.

Why Netlify Functions?

Answer: Thin secure backend for Groq without maintaining Express.

CHECK BEFORE INTERVIEW

- Firestore security rules published in Firebase Console (not just FIRESTORE_SETUP.md).
- GROQ_API_KEY set on Netlify production site.
- Email/Password + Google providers enabled in Firebase Auth.
- Do not claim magic-link send, Hugging Face, MongoDB, Express, or PDF file Storage - not implemented.